

Modèle de données NoSQL - Libreville Eats

1. Description des collections

Le projet s'appuie sur 4 collections MongoDB principales : users, restaurants, commandes et deliverers.

1.1 users

Compte unique pour les quatre rôles de l'application (CUSTOMER, VENDOR, DELIVERER, ADMIN) - un client, un restaurateur et un livreur partagent le même schéma d'authentification.

```
{
  "_id": ObjectId,
  "email": "user@example.com",
  "password": "$2b$10$...",
  "role": "CUSTOMER",
  "profile": {
    "firstName": "Jean",
    "lastName": "Dupont",
    "phone": "+241061234567"
  },
  "addresses": [
    {
      "label": "Home",
      "district": "Batavéa",
      "coordinates": { "type": "Point", "coordinates": [9.4720, 0.4280] },
      "isDefault": true
    }
  ],
  "isActive": true,
  "metadata": { "createdAt": ISODate, "updatedAt": ISODate }
}
```

1.2 restaurants

```
{
  "_id": ObjectId,
  "name": "Chez Albert - Libreville",
  "address": {
    "district": "Quartier Nombakélé",
    "coordinates": { "type": "Point", "coordinates": [9.4583, 0.4162] }
  },
  "isOpen": true,
  "rating": 4.5,
  "menus": [
    {
      "name": "Menu Principal",
      "dishes": [
        { "name": "Poulet Nyembwe", "price": 3500, "category": "Plats Principaux", "isAvailable": true }
      ]
    }
  ],
  "deliveryZones": [
    { "zone": "Quartier Nombakélé", "deliveryFee": 1000, "deliveryTime": 30 }
  ]
}
```

```

],
"owner_id": ObjectId,
"metadata": { "createdAt": ISODate, "totalOrders": 342 }
}

```

1.3 commandes (orders)

```

{
  "_id": ObjectId,
  "customer_id": ObjectId,
  "restaurant_id": ObjectId,
  "deliverer_id": ObjectId,
  "items": [
    { "dishId": ObjectId, "dishName": "Poulet Nyembwe", "quantity": 2, "unitPrice": 3500, "totalPrice": 7000 }
  ],
  "pricing": { "subtotal": 8500, "deliveryFee": 1000, "total": 9500, "currency": "FCFA" },
  "status": "DELIVERY_IN_PROGRESS",
  "statusHistory": [
    { "status": "PENDING", "timestamp": ISODate, "note": "Commande créée" },
    { "status": "CONFIRMED", "timestamp": ISODate, "note": "Restaurant a confirmé" }
  ],
  "deliveryTracking": [
    { "timestamp": ISODate, "coordinates": { "type": "Point", "coordinates": [9.4583, 0.4162] }, "speed": 25.5 }
  ],
  "deliveryInfo": { "type": "DELIVERY", "recipientName": "Jean Dupont" },
  "payment": { "method": "CASH", "status": "PENDING" },
  "metadata": { "createdAt": ISODate, "updatedAt": ISODate }
}

```

1.4 deliverers

```

{
  "_id": ObjectId,
  "user_id": ObjectId,
  "personalInfo": { "firstName": "Ahmed", "lastName": "Makanda", "phone": "+241061234567" },
  "vehicleInfo": { "type": "MOTORCYCLE", "licensePlate": "LA2024ABC" },
  "currentLocation": { "type": "Point", "coordinates": [9.4583, 0.4162], "lastUpdated": ISODate },
  "isActive": true,
  "isAvailable": true,
  "performanceMetrics": { "totalDeliveries": 342, "averageRating": 4.7, "averageDeliveryTime": 28 },
  "metadata": { "createdAt": ISODate }
}

```

2. Relations entre collections

Relation	Type	Cardinalité
`restaurants.owner_id` -> `users._id`	Référence	1 utilisateur -> N restaurants
`deliverers.user_id` -> `users._id`	Référence	1 utilisateur -> 1 livreur
`commandes.customer_id` -> `users._id`	Référence	1 utilisateur -> N commandes
`commandes.restaurant_id` -> `restaurants._id`	Référence	1 restaurant -> N commandes
`commandes.deliverer_id` -> `deliverers._id`	Référence	1 livreur -> N commandes

`restaurants.menus[].dishes[]`	Embedding	1 restaurant -> N menus -> N plats
`commandes.items[]`	Embedding (snapshot)	1 commande -> N items figés au moment de
`commandes.statusHistory[]`	Embedding	1 commande -> N étapes de statut
`commandes.deliveryTracking[]`	Embedding (time-series local)	1 commande -> N positions GPS horodatées
`users.addresses[]`	Embedding	1 utilisateur -> N adresses

Voir schema.png pour le schéma visuel des relations.

3. Choix techniques : embedding vs referencing

Donnée	Choix	Justification
Menus & plats dans `restaurants`	**Embedding**	Toujours lus ensemble (page restaurant),
Zones de livraison dans `restaurants`	**Embedding**	Petite taille fixe, pas de croissance no
`items[]` dans `commandes`	**Embedding (snapshot)**	Le prix/nom du plat au moment de la comm
`statusHistory[]` dans `commandes`	**Embedding**	Audit trail de taille prévisible (8 stat
`deliveryTracking[]` dans `commandes`	**Embedding (time-series local)**	Les requêtes portent presque toujours su
`customer_id`, `restaurant_id`, `delivery_id`	**Referencing**	Ces entités évoluent indépendamment de l
`owner_id` dans `restaurants`	**Referencing**	Un propriétaire peut gérer plusieurs res
`user_id` dans `deliverers`	**Referencing**	Sépare les données d'authentification (v

Règle générale appliquée : on embed quand les données sont (a) lues quasi systématiquement ensemble, (b) de taille bornée et prévisible, (c) peu sujettes à une croissance non corrélée au document parent. On référence quand les entités ont un cycle de vie propre, une cardinalité élevée (N illimité), ou sont partagées entre plusieurs documents parents.

4. Stratégie d'index (résumé)

Voir explain/explain-avant-apres.pdf et scripts/04-index.js pour le détail et la preuve de performance.

Collection	Champs	Type	Usage
`users`	`email`	Unique	Authentification
`users`	`addresses.coordinates`	Géospatial (2dsphere)	Utilisateurs à proximité
`restaurants`	`address.coordinates`	Géospatial (2dsphere)	Recherche par proximité
`restaurants`	`address.district, isOpen`	Composé	Liste restaurants ouverts par quarti
`restaurants`	`name, menus.name, menus.dishes.name`	Textes pondéré	Recherche plein-texte
`commandes`	`status, restaurant_id`	Composé	Commandes actives d'un restaurant (t
`commandes`	`customer_id, createdAt`	Composé	Historique client trié
`commandes`	`deliverer_id, status`	Composé	Suivi des livraisons en cours d'un l
`deliverers`	`currentLocation`	Géospatial (2dsphere)	Livreurs disponibles à proximité